Beyond the Basics

Pseudocode & Python · Flowcharts · Logic Gates

Explanations, worked examples and practice questions with full answers for everything **after** the basics (variables, input/output and decisions). Section 1 builds strong fundamentals with analogies — lockers, pizza, bouncers, two-key safes — and from there pseudocode and Python are treated as equals: every concept and every model answer appears in both languages, with a full section on Python's shortcuts.

Based on the **Cambridge O Level Computer Science 2210** syllabus for 2026–2028, Paper 2: Algorithms, Programming and Logic (Topics 7–10), including the 15-mark scenario question.

Compiled 12 November 2026

Contents

How this guide works 4

1. The fundamentals — how it all works 4

1.1 Variables are labelled boxes . . . 4

1.2 The three kinds of loop — laps, pizza and tea . . . 4

1.3 Accumulators and counters — the till and the clicker . . . 5

1.4 Arrays are corridors of lockers . . . 5

1.5 Strings are necklaces of beads . . . 6

1.6 Procedures and functions — recipe cards and machines . . . 6

1.7 Files are the filing cabinet . . . 6

1.8 Search and sort — the register and the fizzy drink . . . 6

1.9 Trace tables are the flight recorder . . . 6

1.10 Validation is the bouncer; verification is the proofreader . . . 6

1.11 Logic gates are tiny decision machines . . . 7

1.12 A database is a spreadsheet with rules . . . 7

2. Iteration and nested constructs 8

2.1 The three loop types . . . 8

2.2 Totalling and counting . . . 9

2.3 Nested IF and nested loops . . . 10

2.4 Python shortcut — the elif ladder . . . 11

2.5 Loops at a glance . . . 12

2.6 Worked example — trace of a REPEAT loop . . . 12

3. String handling 16

3.1 The four string operations . . . 16

3.2 Processing strings character by character . . . 17

4. Arrays 20

4.1 One-dimensional arrays . . . 20

4.2 Two-dimensional arrays . . . 20

4.3 Python shortcut — for-each . . . 21

5. Procedures, functions and modularity 23

5.1 Why split up code? . . . 23

5.2 Procedures (Python: functions with no return value) . . . 23

5.3 Functions (Python: functions with return) . . . 24

5.4 Local and global variables . . . 24

6. File handling 27

6.1 Why files? . . . 27
6.2 The four file operations . . . 27
6.3 Typical pattern: process a file of values . . . 27
7. Standard algorithms, trace tables and debugging 30
7.1 The seven standard methods . . . 30
7.2 Linear search . . . 30
7.3 Bubble sort . . . 31
7.4 Trace tables and dry-runs . . . 31
8. Program development, testing and data checking 34
8.1 The program development life cycle (PDLC) . . . 34
8.2 Validation — is the data reasonable? . . . 35
8.3 Verification — was it entered accurately? . . . 35
8.4 Test data — normal, abnormal, extreme, boundary . . . 35
9. Flowcharts 36
9.1 The six standard symbols . . . 36
9.2 Loop patterns . . . 37
9.3 Converting between pseudocode and flowcharts . . . 37
10. Boolean logic and logic gates 38
10.1 The seven gates . . . 39
10.2 The three conversions . . . 39
11. Databases 41
11.1 Key concepts . . . 41
11.2 SQL — the examinable keywords . . . 41
11.3 The same query without SQL . . . 42
12. The scenario question — pseudocode and Python as equals 43
12.1 The rules . . . 43
12.2 Strategy for a strong 15-mark answer . . . 43
12.3 Worked mini-scenario . . . 43
1 - collect 10 valid values 46
2 - total and average 46
3 - highest value 46
4 - count days above average 46
Appendix A — pseudocode and Python side-by-side reference 46
Appendix B — command words 47

How this guide works

This guide covers everything **after** the basics — you already know variables, input/output, operators and decisions, so those are skipped. What remains is the heart of Paper 2: iteration, strings, arrays, subroutines, files, standard algorithms, testing, flowcharts, Boolean logic, databases and the scenario question.

Section 1 is different from the rest: it is a fundamentals tour that explains **how and why** every idea works, using analogies (lockers, pizza, bouncers, two-key safes...). If a later section ever feels confusing, go back to section 1 — the analogies are the mental models everything else is built on.

Pseudocode and Python are treated as equals here. Every concept is explained with a pseudocode example **and** a Python example, and every coding question has a model answer in **both** languages.

Common errors and Python shortcuts are woven into each topic, right where they belong: whenever a trap exists, a **Watch out** box sits at the exact point where students fall in; whenever Python offers a tidier form (elif, += and friends), a **Python shortcut** box appears next to the fundamental form it replaces. Learn the long form first — the exam's pseudocode expects it — then use the shortcuts to write cleaner Python.

Each section gives you explanations and worked examples, then practice questions with model answers and mark allocations. Try the questions **before** looking at the answers.

> **Note**
> **The golden exam rule.** In the real Paper 2 exam, every coding answer must be written in **Cambridge pseudocode** — answers in Python (or any real language) score zero, **except** in the final 15-mark scenario question, where pseudocode **or** Python/Visual Basic/Java is allowed. Syntax is never marked: the **logic** is what earns marks. But operators must be precise: write `x > y`, never "x is greater than y".

In this guide, selection is built from **nested IF statements** throughout (in both languages), matching the syllabus requirement for nested selection and iteration. The Python `elif` ladder is introduced as a shortcut inside section 2.3, once the nested form is solid.

1. The fundamentals — how it all works

This section builds the mental models. Each idea gets an analogy you can keep for life, then the same idea in pseudocode and Python. Read it slowly — everything after this is just these models wearing different clothes.

1.1 Variables are labelled boxes

A **variable** is a labelled box on a shelf. The label (the name) never changes; the contents can change any time. `Total ← Total + 1` means: *look inside the box called Total, add 1 to what's there, put the result back in the same box*. The old value is gone — the box holds exactly one thing at a time.

A **constant** is a box with the lid glued shut: `CONSTANT HourlyRate ← 6.50` is set once and never changes. Constants make code read like English and make updates easy (change the value in one place).

```
// pseudocode
Total ← 0
Total ← Total + 5
CONSTANT Rate ← 6.50
```

```
# Python
total = 0
total = total + 5
RATE = 6.50
```

Think of the name as the label and ← / = as "put this into the box".

1.2 The three kinds of loop — laps, pizza and tea

Repetition happens in real life in three flavours. Match the flavour to the loop:

FOR = counting laps on a running track. You decide **before** you start: "10 laps". A lap counter ticks up each time. Because you know the number in advance, there is no question to ask mid-run — you simply go round until the counter passes 10. If you plan 10 laps and realise the track is closed (start > end), you run **zero** laps.

```
// pseudocode                          # Python
FOR I ← 1 TO 10                       for i in range(1, 11):
    OUTPUT I                              print(i)
NEXT I
```

WHILE = eating pizza slices while any remain. Before **every** slice you ask: "is there pizza left?" If the box is empty when you first look, you eat **nothing** — the check happens first. That's why a WHILE body can run zero times.

```
// pseudocode                          # Python
WHILE Slices > 0 DO                   while slices > 0:
    EatSlice                              eat_slice()
    Slices ← Slices - 1                   slices = slices - 1
ENDWHILE
```

REPEAT ... UNTIL = stirring tea until the sugar dissolves. You **always stir at least once** (you can't know before stirring), then check **after** each stir: "dissolved yet?" The loop stops when the answer becomes **TRUE**. Careful: UNTIL asks the **opposite** question to Python's `while` — it stops when the condition *is* true.

```
// pseudocode                          # Python
REPEAT                                while True:
    Stir                                  stir()
UNTIL SugarDissolved = TRUE               if sugar_dissolved == True:
                                              break
```

An **infinite loop** is a hamster wheel: the exit condition never becomes true (often because the counter is never incremented), so the program runs forever. Off-by-one errors are **fence-post errors**: a fence with 10 panels needs 11 posts — counting from 1 to 10 inclusive is **10** numbers, and `TO 10` / `range(1, 11)` gives exactly those.

1.3 Accumulators and counters — the till and the clicker

An **accumulator** is the running total on a till receipt. Each new item is scanned and added to the total already showing. The receipt starts at 0.00 before the first item — that's why `Total ← 0` must come first.

A **counter** is the door clicker a bouncer holds: click once per person entering. In code, `Count ← Count + 1` is one click.

```
// pseudocode                          # Python
Total ← 0                             total = 0
FOR I ← 1 TO 5                        for i in range(1, 6):
    Total ← Total + Price[I]              total = total + price[i - 1]
NEXT I
```

Mixing the two up is a classic error (section 2): adding prices into the counter, or clicking the counter once per pound.

1.4 Arrays are corridors of lockers

An **array** is a corridor of identical lockers with numbers on them. Every locker holds the same kind of thing, and the number (**index**) is how you find a locker fast. Cambridge corridors are usually numbered **1 to N**; Python's are **0 to N−1**. Reading `A[7]` is walking straight to locker 7. Asking for `A[31]` in a corridor of 30 is asking for a locker that doesn't exist — **index out of range** (section 4).

```
// pseudocode                          # Python
DECLARE Marks : ARRAY[1:30] OF INTEGER marks = [0] * 30      # lockers 0..29
Marks[7] ← 95                         marks[6] = 95
OUTPUT Marks[7]                       print(marks[6])
```

A **2D array** is a cinema: rows of seats. `Grid[2, 3]` means *row 2, seat 3*. Nested loops walk the whole cinema: every seat in every row.

1.5 Strings are necklaces of beads

A **string** is a necklace of character beads. `LENGTH(s)` counts the beads. `SUBSTRING(s, 3, 2)` snips a run of 2 beads starting at bead 3. `UCASE` / `LCASE` repaint the beads upper or lower case. Processing a string character by character is sliding a magnifying glass along the necklace one bead at a time — a loop from 1 to `LENGTH(s)`. In Python the beads are numbered from 0, so bead "3" in pseudocode is index 2 in Python (section 3).

1.6 Procedures and functions — recipe cards and machines

A **procedure** is a recipe card on the wall: `CALL` means *let's do the recipe now*. It performs a job (print a line, swap two values) and hands **nothing** back.

A **function** is a kitchen machine (a blender): you put ingredients in (**parameters**), press the button (the call), and it hands you back a smoothie (**return value**). Because a function hands something back, its call sits **inside an expression** — the smoothie replaces the call: `OUTPUT SumSquare(10, 20)`.

Parameters are the ingredients you pass in; the recipe/machine works with whatever it is given. **Local vs global** is notepad vs whiteboard: a variable made inside a subroutine is written in a **personal notepad** that is thrown away afterwards — the **classroom whiteboard** (global) is unchanged. **Decomposition** is Lego: build the castle from small sub-assemblies you can test one at a time.

1.7 Files are the filing cabinet

Variables and arrays live on the "desk" and are swept into the bin when the program ends. A **file** is the school's filing cabinet — data survives between program runs. `OPENFILE ... FOR READ` is taking a folder out to photocopy pages; `FOR WRITE` is putting in a **brand-new empty folder** — whatever was in the old one is gone (that's why writing overwrites). Always `CLOSEFILE`: shut the drawer so the next program finds everything tidy.

1.8 Search and sort — the register and the fizzy drink

Linear search is running your finger down a class register, name by name, until you find "Rahim" — or reach the bottom ("not found").

Bubble sort is a line of kids sorting themselves by height: you walk down the line comparing **neighbours**, swapping any pair out of order. Each full walk "bubbles" the tallest remaining kid to the end of the line — like bubbles rising in a fizzy drink. When a full walk makes **no** swaps, everyone is in order.

1.9 Trace tables are the flight recorder

A **trace table** is an aircraft's flight recorder: at every step of the algorithm, it records the value of every variable and every output. Nothing is hidden — when the algorithm crashes (gives a wrong answer), the table shows exactly which step went wrong. That's how you dry-run an algorithm on paper.

1.10 Validation is the bouncer; verification is the proofreader

Validation is a bouncer at a club door: is this ID **plausible**? Right format, sensible age, actually present? It checks the data makes **sense** — range, length, type, presence, format, check digit.

Verification is a proofreader comparing your typed copy against the original: is it **exactly what was written**? It checks **accuracy of entry** — visual check, double entry.

A **check digit** is the final digit of a credit card or ISBN: it is calculated from the other digits, so mistyping one digit breaks the pattern and the error is caught.

1.11 Logic gates are tiny decision machines

A **logic gate** is a tiny machine with one or two input levers and a light bulb. You know the levers; the bulb lights or not — that's all a gate does.

- **AND** = a two-key safe: **both** keys must turn before it opens.
- **OR** = a house with two doors: **either** door lets you in.
- **NOT** = opposite day: whatever you hear, the gate says the opposite.
- **NAND** / **NOR** = the safe/doors with the wiring reversed: the bulb shows the **opposite** of AND / OR.
- **XOR** = "exactly one of you, not both": the rule you'd use when sharing a pizza with someone who must take one slice and leave the rest.

A **truth table** is simply a list of **every possible combination** of levers — like listing every outcome of flipping two coins (4 rows) or three coins (8 rows). Circuits are just machines bolted together, and the syllabus says you draw them **exactly as described** — no clever shortcuts.

1.12 A database is a spreadsheet with rules

A **database table** is a spreadsheet: each **record** is one row (one student), each **field** is a column (Name, Mark). A **primary key** is the candidate number on an exam script — unique to every student, never reused, never blank. **SQL** is a polite way of asking the spreadsheet questions: *who is in 9A with a mark over 80, highest first?*

Q1. **[3 marks]**

State which loop — FOR, WHILE or REPEAT ... UNTIL — best fits each situation, and justify your choice in one sentence each:
(a) input exactly 20 exam marks;
(b) keep asking for a password until the user types "Open123";
(c) keep offering pizza slices while the box is not empty.

Answer 1

(a) **FOR** — the number of repetitions (20) is known before the loop starts, like planning 20 laps.
(b) **REPEAT ... UNTIL** — the body must run at least once (you must ask at least one time) and it stops when the password becomes correct (condition TRUE).
(c) **WHILE** — the check happens **before** each slice; if the box starts empty, no slices are eaten.
Mark points: one per situation with a fitting justification.

Q2. **[2 marks]**

Using the lockers analogy, explain what `A[7]` does when `A` is declared `ARRAY[1:10] OF INTEGER`, and explain what goes wrong with `A[11]`.

Answer 2

`A[7]` walks straight to the 7th locker in the corridor and reads (or replaces) its contents. `A[11]` asks for an 11th locker, but the corridor only has lockers 1 to 10 — the locker does not exist, so this is an **index out of range** error.
Mark points: correct locker reading for `A[7]` (1); out-of-range explanation for `A[11]` (1).

Q3. **[2 marks]**

For each action, say whether it is **validation** or **verification**, and name the bouncer/proofreader link:
(a) the program checks a typed email address contains an "@" symbol;
(b) the user types the password twice and the program compares the two versions.

Answer 3

(a) **Validation** (bouncer) — a format check that the data looks like an email.
(b) **Verification** (proofreader) — double entry check that the password was typed accurately.
Mark points: one per action with the correct name of the check.

2. Iteration and nested constructs

(The analogies for these are in sections 1.2 and 1.3: FOR = laps, WHILE = pizza, REPEAT = tea, accumulators = the till.)

2.1 The three loop types

Cambridge pseudocode has exactly three loop forms. Choosing the right one (and knowing when each runs **zero**, **one**, or **many** times) is a favourite exam target. Each is shown in both languages.

FOR — count-controlled. Use when you know in advance how many times to repeat.

Pseudocode:

```
FOR I ← 1 TO 10
    OUTPUT I
NEXT I
```

Python:

```
for i in range(1, 11):     # range stops BEFORE 11
    print(i)
```

- The counter is assigned every integer value from the start to the end value **inclusive** (Python's range is exclusive at the end — the classic translation trap).
- If start > end, the body never runs.
- A step size can be given, including negative steps:

Pseudocode:

```
FOR I ← 1 TO 9 STEP 2          // I takes 1, 3, 5, 7, 9
FOR Countdown ← 10 TO 1 STEP -1
```

Python:

```
for i in range(1, 10, 2):           # 1, 3, 5, 7, 9
for countdown in range(10, 0, -1):  # 10, 9, ..., 1
```

Watch out

Fence posts. `FOR I ← 1 TO 5` runs **5** times (1, 2, 3, 4, 5) — not 4. In Python, `range(1, 5)` gives only 1 to 4; to include 5 you need `range(1, 6)`. Whenever a loop "prints one too few", check the fence posts first.

WHILE — pre-condition. The pizza loop: the test happens **before** each run, so the body can run zero times.

Pseudocode:

```
WHILE Password <> "Secret" DO
    INPUT Password
ENDWHILE
```

Python:

```
while password != "Secret":
    password = input("Password: ")
```

Watch out

The hamster wheel. A WHILE loop whose counter is never incremented never ends:

```
I ← 1                              i = 1
WHILE I <= 5 DO                    while i <= 5:
    OUTPUT I                           print(i)
    // forgot I ← I + 1               # forgot i = i + 1
ENDWHILE
```

FOR/NEXT increments for you; in a WHILE loop, **you** must write the increment.

REPEAT ... UNTIL — post-condition. The tea loop: the body runs **at least once**, and the loop **stops when the condition becomes TRUE** — the opposite of Python's `while`.

Pseudocode:

```
REPEAT
    INPUT Password
UNTIL Password = "Secret"       // stops when TRUE
```

Python:

```
while True:
    password = input("Password: ")
    if password == "Secret":
        break                   # stops when TRUE
```

Watch out

UNTIL polarity. `UNTIL` stops when the condition is **TRUE** — the sugar must be dissolved. Writing the wrong side of the condition inverts the loop:

```
REPEAT
    INPUT Password
UNTIL Password <> "Secret"     // WRONG: stops when password is NOT "Secret"
```

The condition must describe the state you want to **stop** on: `UNTIL Password = "Secret"`. In Python the twin mistake is forgetting the `if condition: break` entirely.

2.2 Totalling and counting

Two patterns you will use constantly — the same logic in both languages (the till and the clicker, section 1.3):

Totalling (pseudocode):

```
Total ← 0
FOR I ← 1 TO 5
    INPUT Value
    Total ← Total + Value
NEXT I
OUTPUT Total
```

Totalling (Python):

```
total = 0
for i in range(1, 6):
    value = int(input("Value: "))
    total = total + value
print(total)
```

Counting (pseudocode):

```
Count ← 0
FOR I ← 1 TO 5
    INPUT Value
    IF Value > 10
        THEN
```

```
            Count ← Count + 1
    ENDIF
NEXT I
```

Counting (Python):

```
count = 0
for i in range(1, 6):
    value = int(input("Value: "))
    if value > 10:
        count = count + 1
```

> **Watch out**
> **The till must start at zero.** `Total ← Total + Value` needs `Total ← 0` (and `Count ← 0`) **before** the loop, or the first total is garbage. Also keep the two apart: prices go into the `Total`, clicks go into the `Count` — writing `Count ← Count + Value` runs perfectly and is perfectly wrong.

> **Python shortcut**
> **+= and friends — update the box in place.** `x += 1` is shorthand for `x = x + 1` ("add 1 to the box, store it back in the same box"). The whole family:
> T:
> | Long form | Shortcut |
> | x = x + 1 | x += 1 |
> | x = x - 1 | x -= 1 |
> | x = x * 2 | x *= 2 |
> | x = x / 4 | x /= 4 |
> | total = total + price | total += price |
> Cambridge pseudocode has no += — it always uses the long form (`Total ← Total + Value`) — so write that in exams; in Python, += is the house style. (There are also `x //= 2` and `x %= 3` for integer division and remainder updates.)

2.3 Nested IF and nested loops

The syllabus requires **nested selection and iteration** (never more than **three levels** of nesting). In this guide multi-way choices are built by **nesting IF inside ELSE** — in both languages. Think of nested IF as a **triage of questions**: the second question is only asked if the first one sent you down a particular path.

Nested IF — grade bands. One condition checked inside the other's ELSE branch:

Pseudocode:

```
IF Score >= 80
    THEN
        OUTPUT "Grade A"
    ELSE
        IF Score >= 60
            THEN
                OUTPUT "Grade B"
            ELSE
                OUTPUT "Grade C"
        ENDIF
ENDIF
```

Python (nested if):

```
if score >= 80:
    print("Grade A")
else:
    if score >= 60:
        print("Grade B")
    else:
        print("Grade C")
```

- Each nested IF has its **own** `ENDIF` in pseudocode — count them carefully in error-finding questions.
- In Python the nested IF simply lives **inside the else block** and is indented one level further. Indentation is the structure.
- Three-way choice = one level of nesting; keep it at three levels or fewer.

> **Watch out**
> **Unmatched ENDIF and broken indentation.** In pseudocode, every `IF` needs its own `ENDIF` (two IFs above means two ENDIFs below). In Python, a nested IF left at the wrong indent level silently belongs to the wrong branch — the structure *is* the indentation.

Nested loops — one total per row (the cinema: every seat in every row):

Pseudocode:

```
FOR Row ← 1 TO 3
    RowTotal ← 0
    FOR Column ← 1 TO 4
        RowTotal ← RowTotal + Grid[Row, Column]
    NEXT Column
    OUTPUT "Total for row ", Row, " is ", RowTotal
NEXT Row
```

Python:

```
for row in range(1, 4):
    row_total = 0
    for column in range(1, 5):
        row_total = row_total + grid[row - 1][column - 1]
    print("Total for row", row, "is", row_total)
```

(If the Python grid is 0-based, subtract 1 from the indices — or build the grid 1-based style and keep the indices.)

2.4 Python shortcut — the elif ladder

When there are three or more branches, nesting IF inside ELSE gets deep and repetitive. Python's `elif` (short for "else if") flattens the ladder — the computer treats it **exactly** like the nested version above.

Nested IF (the fundamental form):

```
if score >= 80:
    print("Grade A")
else:
    if score >= 60:
        print("Grade B")
    else:
        if score >= 50:
            print("Grade C")
        else:
            print("Grade U")
```

elif (the shortcut — same logic, one level flatter):

```
if score >= 80:
    print("Grade A")
elif score >= 60:
    print("Grade B")
elif score >= 50:
    print("Grade C")
else:
    print("Grade U")
```

Read `elif` as "otherwise, if...". The conditions are tested **top to bottom**, exactly like the nested spiral: the first true branch wins and the rest are skipped. Cambridge **pseudocode has no elif** — it uses IF ... ELSE ... ENDIF (or CASE OF) — so in exams write the nested form; in Python, prefer elif.

2.5 Loops at a glance

Task	Pseudocode	Python
Count 1 to N	`FOR I ← 1 TO N ... NEXT I`	`for i in range(1, N + 1):`
Count with step S	`FOR I ← A TO B STEP S ... NEXT I`	`for i in range(A, B + 1, S):`
While condition is true	`WHILE c DO ... ENDWHILE`	`while c:`
Repeat at least once, until c is true	`REPEAT ... UNTIL c`	`while True: ... if c: break`
Multi-way choice	nested IF / CASE OF	`elif` ladder

2.6 Worked example — trace of a REPEAT loop

Trace tables (the flight recorder, section 1.9) track every variable and output at each step. For inputs 5, −3, 2, 0:

Pseudocode:

```
Total ← 0
Count ← 0
REPEAT
    INPUT Value
    IF Value > 0
        THEN
            Total ← Total + Value
            Count ← Count + 1
    ENDIF
UNTIL Value = 0
OUTPUT Total, Count
```

Python (same program):

```
total = 0
count = 0
while True:
    value = int(input("Value: "))
    if value > 0:
        total = total + value
        count = count + 1
    if value == 0:
        break
print(total, count)
```

Step	Value	Total	Count
before loop	-	0	0
1	5	5	1
2	-3	5	1
3	2	7	2
4	0	7	2

The loop stops after the input 0 (condition `Value = 0` now TRUE). Final output: `7, 2`.

Q1. **[3 marks]**

Write an algorithm that calculates the sum of the squares of the numbers 1 to 10 (that is 1 + 4 + 9 + ... + 100) and outputs the total. Write it in pseudocode **and** Python. The value 385 must not appear as a literal in either version.

Answer 1

The total is built up with an accumulator (the till, section 1.3) using the square of the loop counter.
Pseudocode:

```
Total ← 0
FOR I ← 1 TO 10
    Total ← Total + I ^ 2
NEXT I
OUTPUT Total
```

Python:

```
total = 0
for i in range(1, 11):
    total = total + i ** 2
print(total)
```

Both output 385. Mark points: initialising `Total` (1); correct loop 1 to 10 and accumulating the square of the counter (1); output of the calculated total (1). Accept `I * I` / `i * i` instead of `I ^ 2` / `i ** 2`.

Q2. **[4 marks]**

The program below is run with inputs 5, −3, 2, 0 in that order. Complete a trace table showing `Value`, `Total` and `Count` at each step, and state what is output.

```
Total ← 0
Count ← 0
REPEAT
    INPUT Value
    IF Value > 0
        THEN
            Total ← Total + Value
            Count ← Count + 1
    ENDIF
UNTIL Value = 0
OUTPUT Total, Count
```

Answer 2

T:
Step	Value	Total	Count
before loop	-	0	0
1	5	5	1
2	-3	5	1
3	2	7	2
4	0	7	2
Output: `7, 2`. The Python twin (see section 2.6) produces exactly the same table and output.
Mark points: `Total` column correct after each input (2); `Count` column correct (1); correct final output (1). Note that −3 is ignored (not > 0) and that 0 is processed before the loop stops, because `UNTIL` is tested after the body.

Q3. **[3 marks]**

Rewrite this FOR loop using a WHILE loop, in pseudocode **and** Python. It must produce exactly the same output.

```
FOR I ← 1 TO 10
    OUTPUT I * 2
NEXT I
```

Answer 3

Pseudocode:

```
I ← 1
WHILE I <= 10 DO
    OUTPUT I * 2
    I ← I + 1
ENDWHILE
```

Python:

```
i = 1
while i <= 10:
    print(i * 2)
    i = i + 1
```

Mark points: initialise the counter (1); correct condition `<= 10` (1); increment and correct WHILE structure (1). Common mistake: using < instead of <=, which prints only 8 values (2 to 16) — the fence-post error of section 1.2.

Q4. **[2 marks]**

This loop is supposed to output the numbers 1 to 5, but it only outputs 1 to 4. Identify the error and write the corrected line. The same bug exists in the Python version — fix it there too.

```
I ← 1
WHILE I < 5 DO
    OUTPUT I
    I ← I + 1
ENDWHILE
```

Answer 4

The condition `I < 5` should be `I <= 5` (in Python: `while i <= 5:`). With < the loop stops when `I` reaches 5, so 5 is never printed — one fence post too few.
Mark points: identifying the wrong condition (1); correcting it in both languages (1).

Q5. **[2 marks]**

Translate this pseudocode into Python. Then state the output of both versions.

```
Total ← 0
FOR I ← 1 TO 9 STEP 2
    Total ← Total + I
NEXT I
OUTPUT Total
```

Answer 5

Python:

```
total = 0
for i in range(1, 10, 2):
    total = total + i
print(total)
```

Both versions print 25 (1 + 3 + 5 + 7 + 9). Mark points: correct loop with `range(1, 10, 2)` (1); accumulation and output (1). Remember `range` stops before its end value: to include 9 the end must be 10.

Q6. **[3 marks]**

This pseudocode is supposed to keep asking until the user enters a valid mark between 0 and 100 inclusive. It does the opposite. Identify the error and fix it.

```
REPEAT
    INPUT Mark
UNTIL Mark < 0 OR Mark > 100
```

Answer 6

UNTIL stops when the condition is **TRUE**, so this loop stops when the mark is **invalid** — it keeps asking while the mark is valid (the UNTIL polarity trap of section 2.1). The condition must describe the state you want to stop on:

```
REPEAT
    INPUT Mark
UNTIL Mark >= 0 AND Mark <= 100
```

Mark points: explaining the UNTIL polarity (1); corrected condition with both bounds and AND (1); correct final code (1). This is the pseudocode twin of the Python "while True / if: break" trap.

Q7. **[3 marks]**

This pseudocode should output the sum 1 + 2 + 3 + 4 + 5 (= 15). Identify the **two** errors and write the corrected code.

```
FOR I ← 1 TO 4
    Total ← Total + I
NEXT I
OUTPUT Total
```

Answer 7

Error 1 — uninitialised accumulator: `Total` is never set to 0 — the till starts at garbage (section 2.2).
Error 2 — off-by-one bound: `TO 4` stops at 4 (fence-post error); the question asks for 1 to 5.

```
Total ← 0
FOR I ← 1 TO 5
    Total ← Total + I
NEXT I
OUTPUT Total
```

Mark points: spotting the missing initialisation (1); spotting the wrong upper bound (1); corrected code outputting 15 (1).

Q8. **[3 marks]**

Rewrite this Python program using the shortcuts from section 2 (there are three to apply).

```
count = 0
for i in range(0, len(marks)):
    mark = marks[i]
    if mark >= 80:
        count = count + 1
        print("Distinction")
    else:
        if mark >= 50:
            count = count + 1
            print("Pass")
        else:
            print("Fail")
print(count)
```

Answer 8

```
count = 0
for mark in marks:
    if mark >= 80:
        count += 1
        print("Distinction")
    elif mark >= 50:
        count += 1
        print("Pass")
    else:
        print("Fail")
print(count)
```

The three shortcuts: **for-each** (`for mark in marks` — see section 4.3), **elif** (section 2.4), and **+=** (section 2.2). The logic is identical — only the typing is shorter.
Mark points: one per shortcut applied correctly.

Q9. **[2 marks]**

State exactly what this Python code prints.

```
n = 5
total = 0
while n > 0:
    total += n
    n -= 1
print(total, n)
```

Answer 9

`15 0`. The loop adds 5 + 4 + 3 + 2 + 1 = 15 and counts n down to 0. `total += n` is `total = total + n` and `n -= 1` is `n = n - 1` — the hamster stops because `n > 0` finally fails.
Mark points: 15 (1); 0 (1).

3. String handling

(Strings are bead necklaces — section 1.5. LENGTH counts the beads; SUBSTRING snips a run; UCASE/LCASE repaint them.)

3.1 The four string operations

Strings are examined heavily because they combine naturally with loops and selection.

Operation	Meaning	Example	Result
LENGTH(s)	number of characters	LENGTH("Happy Days")	10
SUBSTRING(s, start, length)	substring of given length from position start (normally 1 = first character)	SUBSTRING("Happy Days", 1, 5)	"Happy"
UCASE(s)	all characters upper case	UCASE("Happy")	"HAPPY"
LCASE(s)	all characters lower case	LCASE('W')	'w'

The same operations in Python:

Pseudocode	Python	Note
LENGTH(s)	len(s)	same result

Pseudocode	Python	Note
SUBSTRING(s, p, l)	s[p-1 : p-1+l]	Python slices are 0-based
UCASE(s)	s.upper()	
LCASE(s)	s.lower()	

Strings join with + in both languages: `FullName ← FirstName + " " + LastName` / `full_name = first_name + " " + last_name`. Cambridge normally uses 1-based positions — questions will make this clear.

> **Watch out**
> **The offset trap.** Cambridge bead 3 is Python bead 2: `SUBSTRING(S, 3, 3)` = `s[2:5]`. The slicing formula is `s[p-1 : p-1+l]` — subtract 1 from the start **and** add the length to that. When a substring question returns "the wrong letters", check the offset before anything else.

3.2 Processing strings character by character

The standard pattern is a loop over the necklace, bead by bead — count the vowels in S:

Pseudocode:

```
Count ← 0
FOR I ← 1 TO LENGTH(S)
    Ch ← UCASE(SUBSTRING(S, I, 1))
    IF Ch = "A" OR Ch = "E" OR Ch = "I" OR Ch = "O" OR Ch = "U"
        THEN
            Count ← Count + 1
    ENDIF
NEXT I
OUTPUT Count
```

Python:

```
count = 0
for i in range(0, len(s)):
    ch = s[i].upper()
    if ch == "A" or ch == "E" or ch == "I" or ch == "O" or ch == "U":
        count = count + 1
print(count)
```

> **Watch out**
> **Case-sensitive comparisons.** Input "secret" fails against = `"Secret"`. Decide what the question wants: for case-insensitive matching, normalise with UCASE / `.upper()` (as above) or compare against both cases.

Reversing a string — loop downwards along the necklace:

Pseudocode:

```
Reversed ← ""
FOR I ← LENGTH(S) TO 1 STEP -1
    Reversed ← Reversed + SUBSTRING(S, I, 1)
NEXT I
OUTPUT Reversed
```

Python:

```
reversed_text = ""
for i in range(len(s) - 1, -1, -1):
    reversed_text = reversed_text + s[i]
print(reversed_text)
```

Python shortcut

String tricks. The loop above is the fundamental form; Python has neat one-liners built from the same ideas:
- **Membership:** the vowel IF collapses to `if ch in "AEIOU":` — "is this bead in the vowel set?" It works on lists too: `if 90 in marks:`.
- **Reverse slicing:** `print(s[::-1])` reverses the necklace without a loop.
- **Repetition:** `print("-" * 30)` prints 30 dashes — a tidy way to draw a line in outputs.
- **f-strings:** `print(f"Total: {total}")` drops values inside the text — tidier than `"Total: " + str(total)`.

Watch out

Text + number in print. `print("Total: " + total)` is a TypeError — Python refuses to join text to a number. Use `print("Total:", total)`, or `str(total)`, or an f-string.

Other common string tasks: counting occurrences of a character, checking a password against a rule, extracting parts of a formatted code.

Q1. **[3 marks]**

The variable Text contains "Computer". Evaluate:
(a) `LENGTH(Text)` (b) `SUBSTRING(Text, 3, 3)` (c) `UCASE(SUBSTRING(Text, 2, 1))`
Then give the Python equivalent of each expression.

Answer 1

(a) 8 (b) "mpu" (c) "O".
Python equivalents: (a) `len(text)` → 8 (b) `text[2:5]` → "mpu" (c) `text[1:2].upper()` → "O".
Mark points: one per part. Positions: C=1, o=2, m=3, p=4, u=5, t=6, e=7, r=8. `SUBSTRING(Text, 3, 3)` starts at "m" and takes 3 characters; `SUBSTRING(Text, 2, 1)` is "o", upper-cased to "O".
Note the offset: Python index 2 = pseudocode position 3.

Q2. **[4 marks]**

Write an algorithm that inputs a string S and counts how many times the letter "e" appears, in either upper or lower case. Output the count. Write it in pseudocode **and** Python.

Answer 2

Pseudocode:

```
INPUT S
Count ← 0
FOR I ← 1 TO LENGTH(S)
    Ch ← UCASE(SUBSTRING(S, I, 1))
    IF Ch = "E"
        THEN
            Count ← Count + 1
    ENDIF
NEXT I
OUTPUT Count
```

Python:

```
s = input("Text: ")
count = 0
for i in range(0, len(s)):
    ch = s[i].upper()
    if ch == "E":
        count = count + 1
print(count)
```

Mark points: loop over the whole string using LENGTH/len (1); extracting each character (1); case-insensitive comparison (1); counting and outputting (1). Accept comparison against both "e" and "E" instead of converting case.

Q3. **[3 marks]**

Write an algorithm that inputs a string S and outputs it reversed. For input "ABC" the output must be "CBA". Write it in pseudocode **and** Python.

Answer 3

Pseudocode:

```
INPUT S
Reversed ← ""
FOR I ← LENGTH(S) TO 1 STEP -1
    Reversed ← Reversed + SUBSTRING(S, I, 1)
NEXT I
OUTPUT Reversed
```

Python:

```
s = input("Text: ")
reversed_text = ""
for i in range(len(s) - 1, -1, -1):
    reversed_text = reversed_text + s[i]
print(reversed_text)
```

Mark points: loop downwards over the string (1); character extraction (1); concatenation and output (1). In Python, `print(s[::-1])` alone is acceptable if the loop requirement is not being tested.

Q4. **[2 marks]**

This Python code is run. Write down the exact output, then write the pseudocode version of the same program.

```
s = "Computer"
print(len(s), s[2:5], s.upper())
```

Answer 4

Output: `8 mpu COMPUTER` (on one line).
Pseudocode version:

```
S ← "Computer"
OUTPUT LENGTH(S), SUBSTRING(S, 3, 3), UCASE(S)
```

Mark points: 8 and `mpu` (1) — remember Python slices are 0-based, so `s[2:5]` is characters at indexes 2, 3, 4; `COMPUTER` and a correct pseudocode translation (1).

Q5. **[3 marks]**

Write one line of Python for each task, using a shortcut from section 3:
(a) print a line of 30 dashes;
(b) test whether the character `ch` is a vowel, using `in`;
(c) output the string `s` reversed, without a loop.

Answer 5

(a) `print("-" * 30)`
(b) `if ch.upper() in "AEIOU":` (accept `ch in "AEIOUaeiou"`)
(c) `print(s[::-1])`
Mark points: one per task.

4. Arrays

(Lockers and cinema seats — section 1.4. Cambridge lockers are numbered 1 to N; Python's 0 to N−1.)

4.1 One-dimensional arrays

An array is a fixed-length corridor of elements of the **same** data type, accessed by index.

Pseudocode:

```
DECLARE StudentNames : ARRAY[1:30] OF STRING
DECLARE Marks : ARRAY[1:30] OF INTEGER
FOR I ← 1 TO 30
    INPUT Marks[I]
NEXT I
OUTPUT Marks[5]
```

Python:

```
marks = [0] * 30              # 30 slots, indexes 0 to 29
for i in range(0, 30):
    marks[i] = int(input("Mark: "))
print(marks[4])               # the 5th mark
```

In Python a list can also grow: `marks.append(value)`.

Arrays are nearly always used **with loops** — to fill them, print them, search them, or sort them.

Watch out

Lockers that don't exist. For `ARRAY[1:30]` the lockers are `A[1]` to `A[30]`: a loop starting at 0, or ending at 31, walks past the corridor's end (**index out of range**). And the corridors are numbered differently: Cambridge `Marks[5]` is Python `marks[4]` — the same trap as the string offset (section 3.1).

4.2 Two-dimensional arrays

Pseudocode:

```
DECLARE Grid : ARRAY[1:3, 1:4] OF INTEGER
Grid[2, 3] ← 7
OUTPUT Grid[2, 3]
```

Python (list of lists):

```
grid = [[0] * 4 for row in range(3)]   # 3 rows of 4
grid[1][2] = 7                          # row 2, column 3 in 1-based terms
print(grid[1][2])
```

Typical operations need **nested loops**: every seat in every row.

4.3 Python shortcut — for-each

Walking a list by index works, but if you never use the index, let Python hand you each value directly — like walking the corridor taking something out of every locker without reading the numbers:

Long form:

```
total = 0
for i in range(0, len(meals)):
    total = total + meals[i]
```

Shortcut:

```
total = 0
for value in meals:
    total += value
```

Use the indexed loop when you **need** the position (searching, swapping, parallel arrays); use for-each when you just need the values.

Q1. **[3 marks]**

Write an algorithm to declare an array capable of holding 20 integer exam marks, then fill it with values typed by the user. Write it in pseudocode **and** Python.

Answer 1

Pseudocode:

```
DECLARE Marks : ARRAY[1:20] OF INTEGER
FOR I ← 1 TO 20
    INPUT Marks[I]
NEXT I
```

Python:

```
marks = []
for i in range(0, 20):
    marks.append(int(input("Mark: ")))
```

Mark points: declaration of 20 integers (1); loop 20 times (1); input into the element using the loop counter as index (1).

Q2. **[4 marks]**

The array A contains the values 2, 7, 4, 9 in positions 1 to 4. Trace the algorithm below, showing I, A[I] and Max at each step of the loop, and state what is output.

```
Max ← A[1]
FOR I ← 2 TO 4
    IF A[I] > Max
        THEN
            Max ← A[I]
    ENDIF
NEXT I
OUTPUT Max
```

Answer 2

T:
Step	I	A[I]	Max
before loop	-	-	2
1	2	7	7
2	3	4	7
3	4	9	9
Output: 9. The Python twin runs identically (with `a = [2, 7, 4, 9]`, `max_value = a[0]`, `for i in range(1, 4):`).
Mark points: Max starts at 2 (1); each correct row of the trace (3). 7 replaces 2, 4 is ignored, 9 replaces 7.

Q3. **[4 marks]**

A 2D array `Grid` has 2 rows and 3 columns. Row 1 contains 3, 1, 4 and row 2 contains 2, 5, 6. Write an algorithm (pseudocode **and** Python) to output the total of each row. Your program must work for any values stored in the array.

Answer 3

Pseudocode:

```
FOR Row ← 1 TO 2
    Sum ← 0
    FOR Col ← 1 TO 3
        Sum ← Sum + Grid[Row, Col]
    NEXT Col
    OUTPUT "Row ", Row, " total is ", Sum
NEXT Row
```

Python:

```
for row in range(0, 2):
    sum_total = 0
    for col in range(0, 3):
        sum_total = sum_total + grid[row][col]
    print("Row", row + 1, "total is", sum_total)
```

This outputs `Row 1 total is 8` then `Row 2 total is 13`.
Mark points: nested loop covering rows and columns (2); accumulation with both indices (1); correct totals output inside the outer loop (1).

Q4. **[2 marks]**

Show how to build a list/array called `Squares` holding the squares of the numbers 1 to 5 (1, 4, 9, 16, 25) by looping — in both languages.

Answer 4

Pseudocode:

```
DECLARE Squares : ARRAY[1:5] OF INTEGER
FOR I ← 1 TO 5
    Squares[I] ← I ^ 2
NEXT I
```

Python:

```
squares = []
for i in range(1, 6):
    squares.append(i * i)
```

Mark points: correct storage (array declaration / empty list with append) (1); loop producing 1 to 5 and squaring the counter (1). Accept `squares = [i * i for i in range(1, 6)]`.

Q5. **[4 marks]**

The Python program below is meant to total the three scores 50, 70, 90 and print the total. It crashes. Identify the **two** errors and write the corrected program.

```
scores = [50, 70, 90]
total = 0
for i in range(1, 4):
    total = total + scores[i]
print("Total: " + total)
```

Answer 5

Error 1 — 0-based index out of range: `range(1, 4)` visits 1, 2, 3 but valid indexes are 0, 1, 2, so `scores[3]` asks for a locker that doesn't exist (IndexError). Fix: `range(0, 3)` (or `range(len(scores))`).
Error 2 — text plus number: `"Total: " + total` tries to join text to an int (TypeError — the section 3 watch-out). Fix: `print("Total:", total)`.

```
scores = [50, 70, 90]
total = 0
for i in range(0, 3):
    total = total + scores[i]
print("Total:", total)
```

Mark points: each error identified (2 — one per error); each fix shown (2).

5. Procedures, functions and modularity

(Recipe cards, blenders and notepads vs whiteboards — section 1.6.)

5.1 Why split up code?

Topic 7 expects you to understand **decomposition** (the Lego principle — build from small tested sub-assemblies) and Topic 8 expects **maintainable programs**: meaningful identifiers, comments (`//` in pseudocode, `#` in Python), and subroutines. A program built from small procedures and functions is easier to write, test, read and reuse.

5.2 Procedures (Python: functions with no return value)

A procedure is a recipe card: it performs a task and returns nothing. In pseudocode it is defined at the start of the code and called with `CALL`; in Python it is a `def` that is simply named:

Pseudocode:

```
PROCEDURE PrintLine(Symbol : CHAR, Times : INTEGER)
```

```
    DECLARE I : INTEGER
    DECLARE LineText : STRING
    LineText ← ""
    FOR I ← 1 TO Times
        LineText ← LineText + Symbol
    NEXT I
    OUTPUT LineText
ENDPROCEDURE
```

CALL PrintLine('*', 10)

```
**Python:**
```

def print_line(symbol, times):
line_text = ""
for i in range(1, times + 1):
line_text = line_text + symbol
print(line_text)

print_line('*', 10)

```
Parameters are the ingredients handed to the recipe (__Symbol : CHAR__); up to **three** parameters are examine
```

5.3 Functions (Python: functions with return)

A function is the blender: it performs a task and **hands one value back**, so it is used inside expressions — never with `CALL`:

Pseudocode:

```
FUNCTION MaxOf(Value1 : INTEGER, Value2 : INTEGER) RETURNS INTEGER
    IF Value1 > Value2
        THEN
            RETURN Value1
        ELSE
            RETURN Value2
    ENDIF
ENDFUNCTION
```

OUTPUT "The winner is ", MaxOf(Score1, Score2)

```
**Python:**
```

def max_of(value1, value2):
if value1 > value2:
return value1
else:
return value2

print("The winner is", max_of(score1, score2))

```
The pseudocode definition states the return type (__RETURNS INTEGER__); in Python the returned value's type fol
```

> **Watch out**
> **The empty blender.** A Python function without a `return` statement hands back `None` — silently, with no error. The program runs and produces garbage far away from the actual bug. And note the difference: `CALL` is only for pseudocode procedures; a **function** call is an expression — using `CALL` on a function (or `return` in a procedure that should return nothing) is a syntax slip examiners see constantly.

5.4 Local and global variables

A variable declared **inside** a procedure/function is written in a **personal notepad** (local) — it exists only there, and shadows any global of the same name written on the **whiteboard**. Python behaves the same way: assigning to a name inside a function creates a **local**, unless you say `global` (best avoided — return values instead).

Watch out

The shadowing trap. If a subroutine declares a name that already exists globally, it gets a **fresh local** — changes inside the subroutine do not travel back out. If the subroutine is supposed to change the global value, that change must happen through the **return value** (or pseudocode by not re-declaring the variable inside).

Q1. **[3 marks]**

Write a procedure `PrintLine` with parameters `Symbol : CHAR` and `Times : INTEGER` that outputs a single line consisting of `Times` copies of `Symbol`. Then call it to print 10 stars. Provide the Python version too.

Answer 1

Pseudocode:

```
PROCEDURE PrintLine(Symbol : CHAR, Times : INTEGER)
    DECLARE I : INTEGER
    DECLARE LineText : STRING
    LineText ← ""
    FOR I ← 1 TO Times
        LineText ← LineText + Symbol
    NEXT I
    OUTPUT LineText
ENDPROCEDURE
```

CALL PrintLine('*', 10)

```
**Python:**
```

def print_line(symbol, times):
line_text = ""
for i in range(1, times + 1):
line_text = line_text + symbol
print(line_text)

print_line('*', 10)

```
Mark points: correct header with both parameters (1); loop building the string (1); single output of the built
```

Q2. **[4 marks]**

Write a function `IsEven(Number : INTEGER) RETURNS BOOLEAN` that returns `TRUE` when `Number` is even. Then use it in a program that inputs 10 integers and outputs how many are even. Do this in pseudocode **and** Python.

Answer 2

Pseudocode:

```
FUNCTION IsEven(Number : INTEGER) RETURNS BOOLEAN
    RETURN MOD(Number, 2) = 0
ENDFUNCTION
```

DECLARE I : INTEGER
DECLARE Value : INTEGER
DECLARE Count : INTEGER
Count ← 0
FOR I ← 1 TO 10
INPUT Value
IF IsEven(Value)
THEN

Count ← Count + 1
ENDIF
NEXT I
OUTPUT Count

```
**Python:**
```

def is_even(number):
return number % 2 == 0

count = 0
for i in range(1, 11):
value = int(input("Number: "))
if is_even(value):
count = count + 1
print(count)

```
Mark points: function header with return (1); remainder test __MOD(Number, 2) = 0__ / `number % 2 == 0` (1); lo
```

Q3. **[3 marks]**

This program uses a function. State exactly what it outputs, show your working, and give the Python version.

```
FUNCTION Square(X : INTEGER) RETURNS INTEGER
    RETURN X * X
ENDFUNCTION
```

DECLARE N : INTEGER
N ← 3
OUTPUT Square(N) + Square(N + 1)

Answer 3

25. Working: Square(3) = 3 × 3 = 9; Square(4) = 4 × 4 = 16; 9 + 16 = 25.
Python version:

```
def square(x):
    return x * x
```

n = 3
print(square(n) + square(n + 1))

```
Mark points: 9 (1); 16 (1); 25 (1). Function calls are expressions — arguments are evaluated, returned values r
```

Q4. **[2 marks]**

What does this program output? Explain why. Then show the Python equivalent and its output.

```
DECLARE Total : INTEGER
Total ← 5
PROCEDURE AddUp
    DECLARE Total : INTEGER
    Total ← 10
ENDPROCEDURE
CALL AddUp
OUTPUT Total
```

Answer 4

It outputs 5. The `Total` declared inside the procedure is written in the **local notepad**, shadowing the global whiteboard variable; changing it to 10 leaves the global `Total` untouched (the shadowing trap).

Python equivalent:

```
total = 5
```

def add_up():
total = 10 # a new LOCAL variable

add_up()
print(total) # prints 5

```
Mark points: correct output (1); explanation in terms of local vs global variables (1).
```

6. File handling

(The filing cabinet — section 1.7. Desk items are wiped when the program ends; files survive.)

6.1 Why files?

Variables and arrays live on the "desk" and vanish when the program ends. A **file** on backing storage keeps data between program runs — for long-term storage, large data sets, or moving data between programs.

6.2 The four file operations

Pseudocode:

```
OPENFILE "Marks.txt" FOR READ      // or FOR WRITE
READFILE "Marks.txt", Mark
WRITEFILE "Marks.txt", Mark
CLOSEFILE "Marks.txt"
```

Python:

```
f = open("Marks.txt", "r")     # "r" = read  ("FOR READ")
mark = int(f.readline())       # READFILE one line
f.write(str(mark) + "\n")      # WRITEFILE one item/line
f.close()                      # CLOSEFILE
```

("w" = write, like `FOR WRITE`.)

- Open a file in **one mode at a time**.
- `FOR READ` is taking the folder out to copy pages; `FOR WRITE` (Python `"w"`) puts in a **brand-new empty folder** — anything the file contained is destroyed.
- `READFILE` / `WRITEFILE` (or `readline()` / `write()`) move one item at a time into/out of a variable.
- Always `CLOSEFILE` / `f.close()` — shut the drawer.

Watch out

The folder that gets emptied and the drawer left open. Two file errors cause nearly all lost marks: (1) opening `FOR WRITE` on a file whose contents you wanted to **keep** — write mode **erases** first (it does not append); (2) forgetting `CLOSEFILE` / `f.close()`, so later reads find stale or missing data. Also never `WRITEFILE` to a file opened `FOR READ` (or write with `"r"` in Python) — the modes must match the operations.

6.3 Typical pattern: process a file of values

Pseudocode:

```
OPENFILE "Scores.txt" FOR READ
```

```
Total ← 0
FOR I ← 1 TO 10
    READFILE "Scores.txt", Mark
    Total ← Total + Mark
NEXT I
CLOSEFILE "Scores.txt"
OUTPUT "Average is ", Total / 10
```

Python:

```
total = 0
f = open("Scores.txt", "r")
for i in range(10):
    mark = int(f.readline())
    total = total + mark
f.close()
print("Average is", total / 10)
```

Note

The syllabus does not define an "end of file" test, so exam questions say how many items the file holds, or end the data with a sentinel value such as −1. Check the question for which pattern to use.

Q1. **[5 marks]**

The text file "Scores.txt" contains exactly 10 integer exam marks, one per line. Write an algorithm to output the average (mean) of these marks, in pseudocode **and** Python.

Answer 1

Pseudocode:

```
OPENFILE "Scores.txt" FOR READ
Total ← 0
FOR I ← 1 TO 10
    READFILE "Scores.txt", Mark
    Total ← Total + Mark
NEXT I
CLOSEFILE "Scores.txt"
OUTPUT "Average is ", Total / 10
```

Python:

```
total = 0
f = open("Scores.txt", "r")
for i in range(10):
    mark = int(f.readline())
    total = total + mark
f.close()
print("Average is", total / 10)
```

Mark points: open for reading (1); loop 10 times (1); read into a variable and accumulate (1); divide by 10 and output (1); close the file (1).

Q2. **[2 marks]**

State what happens to the existing contents of "Notes.txt" when this statement runs, and why. What is the Python equivalent?

```
OPENFILE "Notes.txt" FOR WRITE
```

Answer 2

The existing contents are **erased/lost** — like replacing a full folder with a brand-new empty one. Opening `FOR WRITE` does not append. The Python equivalent is `f = open("Notes.txt", "w")`, which likewise truncates the file.
Mark points: data is lost/overwritten (1); because `WRITE` mode creates a new file rather than adding to existing data (1).

Q3. **[3 marks]**

A student wants to copy the contents of "Old.txt" into "New.txt". Their code opens "Old.txt" and then writes to it with `WRITEFILE`. Explain what is wrong and give the corrected opening lines in both languages.

Answer 3

A file opened `FOR READ` cannot be written to; also, writing must go to the **destination** file. "Old.txt" must be opened for reading and "New.txt" for writing (two files open at once, one per mode):
Pseudocode:

```
OPENFILE "Old.txt" FOR READ
OPENFILE "New.txt" FOR WRITE
```

Python:

```
f_old = open("Old.txt", "r")
f_new = open("New.txt", "w")
```

Mark points: mode mismatch — cannot write to a file opened for reading (2); correct separate opens (1).

Q4. **[4 marks]**

Write a program that copies a single line of text from "FileA.txt" to "FileB.txt", in pseudocode **and** Python.

Answer 4

Pseudocode:

```
DECLARE LineOfText : STRING
OPENFILE "FileA.txt" FOR READ
OPENFILE "FileB.txt" FOR WRITE
READFILE "FileA.txt", LineOfText
WRITEFILE "FileB.txt", LineOfText
CLOSEFILE "FileA.txt"
CLOSEFILE "FileB.txt"
```

Python:

```
f_a = open("FileA.txt", "r")
f_b = open("FileB.txt", "w")
line_of_text = f_a.readline()
f_b.write(line_of_text)
f_a.close()
f_b.close()
```

Mark points: both files opened in correct modes (1); read one line (1); write one line (1); both closed (1).

7. Standard algorithms, trace tables and debugging

(The register scan and the fizzy drink — section 1.8. The flight recorder — section 1.9.)

7.1 The seven standard methods

The syllabus limits "standard methods of solution" to exactly these — you must be able to **write**, **trace** and **apply** each one in both languages:

- **Linear search** — find a value in an array
- **Bubble sort** — sort an array into order
- **Totalling** — add up values (section 2)
- **Counting** — count values matching a condition (section 2)
- **Finding the maximum** — largest value
- **Finding the minimum** — smallest value
- **Finding the average** — total ÷ count

Maximum (minimum works the same with < / <):

Pseudocode:

```
Max ← A[1]
FOR I ← 2 TO 12
    IF A[I] > Max
        THEN
            Max ← A[I]
    ENDIF
NEXT I
```

Python:

```
max_value = a[0]
for i in range(1, 12):
    if a[i] > max_value:
        max_value = a[i]
```

> **Python shortcut**
> **sum, max, min, len.** Python hands you these results directly: `total = sum(meals)`, `biggest = max(meals)`, `smallest = min(meals)`, `how_many = len(meals)`. They are legitimate, readable Python for the scenario question. But be careful: if an exam question asks you to **write** the standard method (find the maximum, total the values), write the loop above — the method itself is what's being tested.

7.2 Linear search

Run your finger down the register, comparing each name to the search key; stop when found:

Pseudocode:

```
Found ← FALSE
Position ← 0
I ← 1
WHILE Found = FALSE AND I <= 4 DO
    IF A[I] = SearchKey
        THEN
            Found ← TRUE
            Position ← I
    ENDIF
    I ← I + 1
ENDWHILE
IF Found = TRUE
    THEN
        OUTPUT "Found at position ", Position
    ELSE
        OUTPUT "Not found"
```

```
ENDIF
```

Python (positions 0-based):

```
found = False
position = 0
i = 0
while found == False and i < 4:
    if a[i] == search_key:
        found = True
        position = i
    i = i + 1
if found == True:
    print("Found at position", position)
else:
    print("Not found")
```

Note the index bases: pseudocode position 1..4 with `<= 4`; Python index 0..3 with `< 4`.

7.3 Bubble sort

Repeatedly walk the line of kids, swapping neighbouring pairs that are out of order. Each walk floats the next tallest "bubble" to its place at the end:

Pseudocode:

```
FOR Outer ← 1 TO N - 1
    FOR Inner ← 1 TO N - Outer
        IF A[Inner] > A[Inner + 1]
            THEN
                Temp ← A[Inner]
                A[Inner] ← A[Inner + 1]
                A[Inner + 1] ← Temp
        ENDIF
    NEXT Inner
NEXT Outer
```

Python:

```
for outer in range(0, n - 1):
    for inner in range(0, n - 1 - outer):
        if a[inner] > a[inner + 1]:
            temp = a[inner]
            a[inner] = a[inner + 1]
            a[inner + 1] = temp
```

> **Watch out**
> **The neighbour beyond the end.** The inner loop compares `A[Inner]` with `A[Inner + 1]`, so it must stop **one locker early**: going `TO N` reads `A[N + 1]` (index out of range — Python raises IndexError). The bound is `N - Outer` (or at least `N - 1`); in Python, `range(0, n)` is wrong for the same reason.

Worked trace sorting 5, 3, 4 (ascending):

Pass	Comparison	Swap?	Array after step
1	5 vs 3	yes	3, 5, 4
1	5 vs 4	yes	3, 4, 5
2	3 vs 4	no	3, 4, 5
2	4 vs 5	no	3, 4, 5

Two "no swap" comparisons in a row mean the array is sorted.

7.4 Trace tables and dry-runs

A trace table is the flight recorder (section 1.9). Give **one column per variable** (plus one for outputs/prompts) and **one row per step** — usually one row per loop iteration. Exam tips:

- Write down the initial values row first.
- Show the row **after** each iteration's changes.
- If asked for outputs, add an OUTPUT column and write each output when it happens.
- Keep types consistent (don't put "TRUE" in a number column).
- A trace table works identically for the pseudocode and Python versions of a program — the logic is the same.

The **Watch out** boxes in sections 2 to 6 are the debugging checklist: hamster wheel (missing increment), fence posts (off-by-one), UNTIL polarity, uninitialised till, offset traps, lockers that don't exist, empty blender, mode mismatch. When a program misbehaves, run down that list before anything else.

Q1. **[4 marks]**

The array A holds 7, 2, 9, 4 in positions 1 to 4 and SearchKey is 9. Complete a trace table for the linear search below (columns `I`, `A[I]`, `Found`, `Position`) and state the output.

```
Found ← FALSE
Position ← 0
I ← 1
WHILE Found = FALSE AND I <= 4 DO
    IF A[I] = SearchKey
        THEN
            Found ← TRUE
            Position ← I
    ENDIF
    I ← I + 1
ENDWHILE
OUTPUT Position
```

Answer 1

T:
Iteration	I	A[I]	Found	Position
before loop	1	-	FALSE	0
1	1	7	FALSE	0
2	2	2	FALSE	0
3	3	9	TRUE	3
Output: 3 (the Python twin prints 2 — its positions are 0-based!). The loop stops once Found becomes TRUE, so iteration 4 never runs.
Mark points: three correct iterations (3); correct output 3 (1).

Q2. **[5 marks]**

Trace one complete bubble sort of the array `5, 3, 4` (ascending order). Show the array after every comparison and swap, and say how you know the sort is finished.

Answer 2

T:
Pass	Comparison	Action	Array after step
1	5 vs 3	swap	3, 5, 4
1	5 vs 4	swap	3, 4, 5
2	3 vs 4	no swap	3, 4, 5
2	4 vs 5	no swap	3, 4, 5
The sort is finished because pass 2 performs no swaps at all — every pair is already in order. Final array: 3, 4, 5. The pseudocode of section 7.3 and its Python twin produce this identical trace.
Mark points: first swap correct (2); second swap correct (1); pass 2 with no swaps (1); final array / reasoning (1).

Q3. **[5 marks]**

Write an algorithm that inputs 12 integers and outputs the **largest** and the **smallest** of them, in pseudocode **and** Python.

Answer 3

Pseudocode:

```
INPUT Value
Largest ← Value
Smallest ← Value
FOR I ← 2 TO 12
    INPUT Value
    IF Value > Largest
        THEN
            Largest ← Value
    ENDIF
    IF Value < Smallest
        THEN
            Smallest ← Value
    ENDIF
NEXT I
OUTPUT "Largest is ", Largest
OUTPUT "Smallest is ", Smallest
```

Python:

```
value = int(input("Number: "))
largest = value
smallest = value
for i in range(2, 13):
    value = int(input("Number: "))
    if value > largest:
        largest = value
    if value < smallest:
        smallest = value
print("Largest is", largest)
print("Smallest is", smallest)
```

Mark points: initialising both with the first value (1); loop covering the remaining 11 values (1); update largest (1); update smallest (1); both outputs (1).

Q4. **[3 marks]**

This bubble sort should sort A[1] to A[N], but it sometimes crashes. Identify the error and write the corrected inner loop line — then show the same bug in Python form.

```
FOR Outer ← 1 TO N - 1
    FOR Inner ← 1 TO N
        IF A[Inner] > A[Inner + 1]
            THEN
                Temp ← A[Inner]
                A[Inner] ← A[Inner + 1]
                A[Inner + 1] ← Temp
        ENDIF
    NEXT Inner
NEXT Outer
```

Answer 4

When `Inner = N` the code reads `A[N + 1]`, which is **outside the array** — asking for a locker that doesn't exist (index out of range). The inner loop must stop one element early:

```
    FOR Inner ← 1 TO N - Outer
```

(`TO N - 1` is also acceptable.) The same bug in Python:

```
for inner in range(0, n):          # WRONG - reads a[n] on the last pass
    if a[inner] > a[inner + 1]:    # IndexError when inner == n - 1 ... a[n]
```

Fixed: `for inner in range(0, n - 1 - outer):`.
Mark points: identifying the out-of-range read of `A[N + 1]` (2); correcting the loop bound (1).

Q5. **[2 marks]**

Explain why programmers use dry-runs and trace tables before running a program.

Answer 5

They verify the **logic** of an algorithm step by step without executing it — the flight recorder shows every variable's value at every step, which reveals errors (wrong conditions, uninitialised variables, bad arithmetic) early, before coding or when no computer is available. The same dry-run works for pseudocode and Python, because the logic — not the syntax — is what is being checked.
Mark points: checking logic / finding errors before running (1); detail about tracking values at each step (1).

8. Program development, testing and data checking

(The bouncer and the proofreader — section 1.10.)

8.1 The program development life cycle (PDLC)

Four stages — know what happens in **each** and which tasks belong where:

Stage	What happens
Analysis	Identify the problem and requirements; abstraction (focus on essentials); decomposition into parts
Design	Decomposition into sub-systems; structure diagrams; flowcharts; pseudocode
Coding	Write the program code; test iteratively as you go

Stage	What happens
Testing	Run the finished code against test data; find and fix remaining errors

8.2 Validation — is the data reasonable?

Validation is the **bouncer** (section 1.10): checks run **automatically on input** to reject impossible or suspicious values before processing. The six check types in the syllabus:

- **Range check** — is a numeric value within limits? (e.g. age between 13 and 19)
- **Length check** — does a value have the right number of characters? (e.g. 6-character username)
- **Type check** — is it the right kind of data? (e.g. a number where a number is expected)
- **Presence check** — has anything been entered at all?
- **Format check** — does it match the expected pattern? (e.g. letters only)
- **Check digit** — an extra digit calculated from the others (like the last digit of a credit card — section 1.10); recalculated on receipt and compared to detect mistyping

A range check in code, both languages:

```
// pseudocode
IF Age < 13 OR Age > 19
    THEN
        OUTPUT "Invalid age"
ENDIF
```

```
# Python
if age < 13 or age > 19:
    print("Invalid age")
```

8.3 Verification — was it entered accurately?

Verification is the **proofreader** (section 1.10): it checks that data was **transferred/recorded correctly** (as opposed to being reasonable):

- **Visual check** — a person re-reads the entered data against the source.
- **Double entry check** — the same data is entered twice and the two entries compared; a mismatch is flagged.

8.4 Test data — normal, abnormal, extreme, boundary

Testing needs data chosen to prove the program handles every case — like a crash-test dummy for each kind of impact:

- **Normal** — typical valid value that should be accepted (e.g. 15).
- **Abnormal** — invalid data that must be rejected (e.g. "abc" typed for a number, or −5 for an age).
- **Extreme** — the **largest and smallest acceptable** values (13 and 19).
- **Boundary** — the extreme values **plus** the corresponding values that must be rejected: just outside on each side (12 and 20).

Watch out

Boundary data catches fence posts. Boundary values are where programs most often fail — the off-by-one errors of section 2.1 (< instead of <=, range stopping one step early). A good test plan always includes boundary values for every range.

Q1. **[3 marks]**

A registration program requires a username that is **exactly 6 characters long** and contains **letters only**. Identify the validation checks needed and say what each one checks.

Answer 1

- **Presence check** — that a username has been entered at all.
- **Length check** — that the username is exactly 6 characters (`LENGTH(Username) = 6` / `len(username) == 6`).
- **Format check** — that it contains letters only (no digits or symbols).

Mark points: one per check, with a correct description of what it checks.

Q2. **[4 marks]**

An online competition form accepts only ages between **13 and 19 inclusive**. Suggest suitable test data of each type: (a) normal (b) abnormal (c) extreme (d) boundary.

Answer 2

(a) Normal: 15 (any valid age such as 13–19).
(b) Abnormal: "abc" (a word where a number is expected). Accept any clearly invalid value, e.g. −5 or 25.
(c) Extreme: 13 and 19 — the smallest and largest **acceptable** values.
(d) Boundary: 12, 13, 19, 20 — the extreme values plus the smallest and largest **rejected** values.
Mark points: one per category, with extreme and boundary giving values on **both** sides of the range.

Q3. **[3 marks]**

Classify each task by PDLC stage (analysis, design, coding or testing):
(a) Identifying what data the program must store (b) Drawing a structure diagram (c) Running the program with sample data to find errors

Answer 3

(a) Analysis (b) Design (c) Testing.
Mark points: one per task.

Q4. **[2 marks]**

Define **validation** and **verification**, making the difference between them clear.

Answer 4

Validation checks that input data is **reasonable/acceptable** (within range, right type/format) before it is processed — the bouncer checking the ID looks real. **Verification** checks that data has been **transferred or entered accurately** — the proofreader checking the copy matches the original. One checks sense; the other checks accuracy of entry.
Mark points: validation definition (1); verification definition (1).

9. Flowcharts

9.1 The six standard symbols

You must draw flowcharts with **exactly** these shapes — examiners penalise substitutes:

Symbol	Shape	Meaning
Terminator	Rounded box (stadium)	Start and Stop of the process

Symbol	Shape	Meaning
Process	Rectangle	Something done, e.g. Total ← Total + Value
Decision	Diamond	A Yes/No (True/False) question with exactly two exit lines
Input/Output	Parallelogram	INPUT or OUTPUT of data
Subroutine	Rectangle with double side bars	Call to a separate, non-linked flowchart
Flow line	Arrow	Control passing between shapes

A flowchart is the algorithm drawn as a **road map**: boxes are places you visit, arrows are the roads, and a diamond is a fork in the road where the sign asks a yes/no question.

9.2 Loop patterns

There is no "loop box" — loops are a **road that circles back**. A WHILE-style loop has the decision before the body; a REPEAT-style loop has it after (its diamond usually sits at the bottom).

Worked example — the password loop as an ASCII sketch (real answers are drawn with the shapes above):



The same loop in code — pseudocode and Python as equals:

```
// pseudocode
REPEAT
    INPUT Password
UNTIL Password = "Secret"
OUTPUT "Granted"
```

```
# Python
while True:
    password = input("Password: ")
    if password == "Secret":
        break
print("Granted")
```

9.3 Converting between pseudocode and flowcharts

- One box per action; keep assignments (← / =) inside **Process** boxes.
- Conditions go in **Decision** diamonds; label both exits (Yes/No or True/False).
- Initialise counters/accumulators **before** the loop in their own Process box.
- A FOR loop becomes: initialiser → decision (counter within range?) → body → increment → flow line back to the decision.

Q1. **[4 marks]**

Draw a flowchart for this process: *Start. Repeatedly ask for a password until the user enters "Secret". Then output "Access granted" and stop.* (Describe each box, its shape, and where every flow line goes.)

Answer 1

- **Terminator** (rounded): "Start".
- **Input/Output** (parallelogram): "INPUT Password".
- **Decision** (diamond): "Password = 'Secret'?" — exit lines labelled **No** and **Yes**.
- The **No** line loops back to the flow line above the Input/Output box.
- The **Yes** line goes to an **Input/Output** (parallelogram): "OUTPUT 'Access granted'".
- **Terminator** (rounded): "Stop".

Mark points: correct shapes for terminator, input/output and decision (2); No branch looping back to the input (1); Yes branch with output and stop, labels present (1).

Q2. **[4 marks]**

Describe the flowchart for this pseudocode, naming each box's shape and its contents, and saying where the loop line goes.

```
Total ← 0
FOR I ← 1 TO 3
    INPUT Value
    Total ← Total + Value
NEXT I
OUTPUT Total
```

Answer 2

- **Terminator**: "Start".
- **Process** box: "Total ← 0", "I ← 1".
- **Decision** diamond: "I <= 3?" with a **Yes** exit going down and a **No** exit going towards the output.
- Yes path: **Input/Output** "INPUT Value" → **Process** "Total ← Total + Value" → **Process** "I ← I + 1" → flow line back up to the **Decision**.
- No path: **Input/Output** "OUTPUT Total" → **Terminator** "Stop".

Mark points: initialisation in a process box (1); decision with `I <= 3` and a loop-back line (1); body boxes in the correct order (1); No path with output and stop (1).

Q3. **[2 marks]**

Sam drew a flowchart in which the decision was drawn as a parallelogram, and one exit of the decision had no label. State what is wrong in each case and how to fix it.

Answer 3

(a) A decision must be drawn as a **diamond** — a parallelogram is the Input/Output shape. (b) Both exit lines of a decision must be labelled (Yes/No or True/False) so the reader knows which condition leads where.

Mark points: one per error with the correction.

10. Boolean logic and logic gates

(The tiny decision machines — section 1.11: AND is the two-key safe, OR is the two-door house, XOR is "one slice each, not both".)

10.1 The seven gates

Logic values: TRUE = 1, FALSE = 0. The syllabus examines **NOT, AND, OR, NAND, NOR, XOR (also called EOR)**. NOT has a **single** input; all others have exactly **two**. Circuits are limited to **three inputs and one output**, drawn **exactly as the statement describes — never simplified**.

A	B	AND	OR	NAND	NOR	XOR
0	0	0	0	1	1	0
0	1	0	1	1	0	1
1	0	0	1	1	0	1
1	1	1	1	0	0	0

- **NOT** inverts its input: 0 → 1, 1 → 0 (opposite day).
- **NAND** = NOT AND: the two-key safe with the wiring reversed — the bulb is off only when both keys turn.
- **NOR** = NOT OR: the two-door house with the wiring reversed — the bulb is on only when both doors are shut.
- **XOR** outputs 1 only when the inputs are **different**.

10.2 The three conversions

You must convert freely in every direction between **problem statement**, **logic expression**, **truth table** and **logic circuit**. Worked example: *A greenhouse fan (X) runs when the temperature is high (A) AND the window is open (B), OR whenever the manual switch (C) is on.*

Expression: `X = (A AND B) OR C`

Circuit: an **AND** gate (the two-key safe: heat **and** open window) whose output feeds an **OR** gate together with input C (either the safe or the manual switch can start the fan).

A	B	C	A AND B	X = (A AND B) OR C
0	0	0	0	0
0	0	1	0	1
0	1	0	0	0
0	1	1	0	1
1	0	0	0	0
1	0	1	0	1
1	1	0	1	1
1	1	1	1	1

The truth table lists **every combination** of the three levers — like every outcome of flipping three coins (8 rows).

The same logic expressed in code — equals in both languages:

```
// pseudocode
INPUT A
INPUT B
INPUT C
X ← (A AND B) OR C
OUTPUT X
```

```
# Python (0 is False, 1 is True)
a = int(input("A: "))
b = int(input("B: "))
c = int(input("C: "))
x = (a and b) or c
print(int(x))
```

Watch out

Parentheses and precedence. Write expressions with explicit parentheses: `(A AND NOT B) OR C`. Pseudocode has no operator precedence rule for mixed logical operators, and examiners want the logic to be unambiguous. And count the rows: a 3-input truth table always has **8 rows** (000 to 111) — a table with 6 rows is missing combinations.

Q1. **[3 marks]**

Complete the truth table for `X = (A OR B) AND NOT C`.

Answer 1

T:

A	B	C	A OR B	NOT C	X
0	0	0	0	1	0
0	0	1	0	0	0
0	1	0	1	1	1
0	1	1	1	0	0
1	0	0	1	1	1
1	0	1	1	0	0
1	1	0	1	1	1
1	1	1	1	0	0

Output column: `0, 0, 1, 0, 1, 0, 1, 0`.

Mark points: correct intermediate columns (1); correct X column (2). Accept X column alone for 3 marks.

Q2. **[3 marks]**

Write a logic expression (and name the gate) for: *X is TRUE only when exactly one of the inputs A and B is TRUE.*

Answer 2

`X = (A AND NOT B) OR (B AND NOT A)`. This is the **XOR** gate (the "one slice each, not both" gate), so `X = A XOR B` is also correct.

Mark points: correct expanded expression with AND, NOT and OR (2); correct parentheses (1). Full marks for `X = A XOR B` (or `A EOR B`).

Q3. **[3 marks]**

Draw the logic circuit for `X = A AND NOT B`. (Describe the gates and connections.)

Answer 3

A **NOT** gate with input B. An **AND** gate whose two inputs are A (direct) and the output of the NOT gate. The AND gate's output is X.

Mark points: identifying the AND gate (1); identifying the NOT gate on B (1); correct connections (1).

Q4. **[4 marks]**

Complete the truth table for `X = (A AND B) OR (NOT C)`.

Answer 4

T:
A	B	C	A AND B	NOT C	X
0	0	0	0	1	1
0	0	1	0	0	0
0	1	0	0	1	1
0	1	1	0	0	0
1	0	0	0	1	1
1	0	1	0	0	0
1	1	0	1	1	1
1	1	1	1	0	1
Output column: `1, 0, 1, 0, 1, 0, 1, 1`.
Mark points: one mark per pair of correct rows (4).

Q5. **[2 marks]**

Compare the behaviour of the OR and XOR gates.

Answer 5

OR outputs 1 when **one or both** inputs are 1 (1 OR 1 = 1) — either of two doors lets you in, even both. **XOR** outputs 1 only when the inputs are **different** (1 XOR 1 = 0) — one slice each, not both. They differ only in the case where both inputs are 1.
Mark points: OR behaviour (1); XOR behaviour including the both-1 case (1).

11. Databases

(The spreadsheet with rules — section 1.12: records are rows, fields are columns, the primary key is the candidate number.)

11.1 Key concepts

- A **table** holds **records** (rows) and **fields** (columns).
- Field data types: **text/alphanumeric**, **character**, **Boolean**, **integer**, **real**, **date/time**.
- A **primary key** is a field (or combination) that **uniquely identifies each record** — the candidate number on every exam script: no duplicates, no empty values, never changes (e.g. `StudentID`).

11.2 SQL — the examinable keywords

SELECT · FROM · WHERE · ORDER BY ASCENDING · ORDER BY DESCENDING · SUM · COUNT · AND · OR

All questions use a **single table**. Worked example — table `Students`:

StudentID	Name	Class	Mark
101	Aisha	9A	84
102	Ben	9B	72
103	Chen	9A	91
104	Dina	9B	65
105	Elias	9A	78

```
SELECT Name FROM Students WHERE Class = "9A" ORDER BY Mark DESCENDING
```

Output: Chen (91), Aisha (84), Elias (78) — only 9A rows, highest mark first.

```
SELECT COUNT(StudentID) FROM Students WHERE Mark >= 80 AND Class = "9A"
```

Output: 2 (Aisha and Chen).

```
SELECT SUM(Mark) FROM Students WHERE Class = "9B"
```

Output: 137 (72 + 65).

> **Watch out**
> **Every row, both conditions.** When predicting SQL output, check each record against **all** the WHERE conditions (AND means both must hold). Missing one row — or letting a record pass on only one condition — is the classic COUNT mistake.

11.3 The same query without SQL

A query is just selection and processing — the same logic in pseudocode and Python (using parallel arrays of the table's columns):

Pseudocode:

```
FOR I ← 1 TO 5
    IF Class[I] = "9A" AND Mark[I] >= 80
        THEN
            OUTPUT Name[I]
    ENDIF
NEXT I
```

Python:

```
for i in range(0, 5):
    if class_name[i] == "9A" and mark[i] >= 80:
        print(name[i])
```

(Ordering by mark first would use the bubble sort of section 7.3.)

Q1. **[3 marks]**

Write an SQL query to list the `Name` of every student in class "9A" with a `Mark` of 80 or more, showing the highest mark first.

Answer 1

```
SELECT Name FROM Students WHERE Class = "9A" AND Mark >= 80 ORDER BY Mark DESCENDING
```

Mark points: `SELECT Name FROM Students` (1); `WHERE` combining both conditions with `AND` (1); `ORDER BY Mark DESCENDING` (1).

Q2. **[3 marks]**

Using the `Students` table above, state the exact output of:

```
SELECT COUNT(StudentID) FROM Students WHERE Mark >= 70 AND Class = "9A"
```

Answer 2

3. The records matching **both** conditions are Aisha (84), Chen (91) and Elias (78) — each is in 9A **and** has a mark of at least 70. Ben has 72 but is in 9B, and Dina's 65 is below 70.
Mark points: correct output 3 (2); correctly identifying the matching records (1).

Q3. **[2 marks]**

A school table has the fields `Name`, `DateOfBirth`, `Class` and `StudentID`. Suggest a suitable primary key and justify your choice.

Answer 3

`StudentID` — it is unique to every student (unlike Name or Class), always has a value, and never changes.
Mark points: correct field (1); justification in terms of uniqueness/stability (1).

12. The scenario question — pseudocode and Python as equals

12.1 The rules

The **last question of Paper 2** is a 15-mark unseen scenario (~30 minutes of your exam). You may answer in **pseudocode, Python, Visual Basic or Java** — every other question on the paper must be pseudocode. Whichever language you choose, the **logic** earns the marks (structure, correctness, completeness), not language-specific cleverness. Write comments and meaningful variable names — the marking rewards **maintainable** code. In Python answers, the shortcuts from sections 2 to 4 (elif, +=, for-each, f-strings) make your code shorter and cleaner without changing the logic.

12.2 Strategy for a strong 15-mark answer

- **Read the scenario twice.** Underline the inputs, outputs, processing and any validation rules — this is the analysis stage and it prevents losing easy marks.
- **Plan on the question paper**: name your variables/arrays, sketch the main loop, decide whether to use a function or two (e.g. a validation function, a `FindHighest` function).
- **Cover every requirement**: each bullet of the question maps to marks. A missing feature is missing marks.
- **Validate inputs** where the scenario implies limits, and handle the "not found"/empty case where relevant.
- **Use meaningful identifiers and comments** (the syllabus explicitly assesses maintainability).
- **Sanity-check with test data** mentally (a normal value, a boundary value) before moving on.

12.3 Worked mini-scenario

Ask the user for 5 daily temperatures. Store them. Output the average and the number of days above 25 degrees.

Pseudocode:

```
DECLARE Temps : ARRAY[1:5] OF REAL
FOR Day ← 1 TO 5
    INPUT Temps[Day]
NEXT Day
Total ← 0
FOR I ← 1 TO 5
    Total ← Total + Temps[I]
NEXT I
Average ← Total / 5
OUTPUT "Average temperature: ", Average
HotDays ← 0
FOR I ← 1 TO 5
    IF Temps[I] > 25
        THEN
            HotDays ← HotDays + 1
    ENDIF
NEXT I
OUTPUT "Days above 25 degrees: ", HotDays
```

Python (with the shortcuts of sections 2 to 4):

```
# Store 5 daily temperatures and report the average and hot days
temps = []
for day in range(1, 6):
    value = float(input("Temperature for day " + str(day) + ": "))
    temps.append(value)
```

total = 0
for value in temps:
total += value
average = total / 5
print(f"Average temperature: {average}")

hot_days = 0
for value in temps:
if value > 25:
hot_days += 1
print(f"Days above 25 degrees: {hot_days}")

Q1. **[10 marks]**

Scenario practice. The school canteen records how many meals were sold each day for 10 days. Write a complete program that:

- asks the user to enter the number of meals sold for each of the 10 days, rejecting and re-asking on any **negative** value;
- stores the 10 values;
- outputs the **total** and the **average** number of meals sold;
- outputs the **highest** number of meals sold;
- outputs **how many days** were above the average.

Write your solution in **either** pseudocode or Python (in the exam, either is allowed for this question only). Model answers for both are below.

Answer 1

Model answer — pseudocode:

```
FUNCTION GetMeals(Day : INTEGER) RETURNS INTEGER
    REPEAT
        OUTPUT "Meals sold on day ", Day
        INPUT Value
        IF Value < 0
            THEN
                OUTPUT "Invalid: the number must not be negative"
        ENDIF
    UNTIL Value >= 0
    RETURN Value
ENDFUNCTION
```

DECLARE Meals : ARRAY[1:10] OF INTEGER
DECLARE Day : INTEGER
FOR Day ← 1 TO 10
Meals[Day] ← GetMeals(Day)
NEXT Day

Total ← 0
FOR Day ← 1 TO 10
Total ← Total + Meals[Day]
NEXT Day
Average ← Total / 10
OUTPUT "Total meals sold: ", Total
OUTPUT "Average per day: ", Average

```
Highest ← Meals[1]
FOR Day ← 2 TO 10
IF Meals[Day] > Highest
THEN
Highest ← Meals[Day]
ENDIF
NEXT Day
OUTPUT "Highest day: ", Highest

Above ← 0
FOR Day ← 1 TO 10
IF Meals[Day] > Average
THEN
Above ← Above + 1
ENDIF
NEXT Day
OUTPUT "Days above average: ", Above
```

```
 **Model answer — Python (with the shortcuts):**

def get_meals(day):
"""Ask for one day's sales, rejecting negative values."""
while True:
value = int(input(f"Meals sold on day {day}: "))
if value >= 0:
return value
print("Invalid: the number must not be negative")

def find_highest(values):
"""Return the largest value in the list."""
highest = values[0]
for v in values:
if v > highest:
highest = v
return highest
```

1 - collect 10 valid values

2 - total and average

3 - highest value

4 - count days above average

Appendix A — pseudocode and Python side-by-side reference

Pseudocode	Python
DECLARE X : INTEGER	x = 0 (just name it; comment the type)
CONSTANT Rate ← 5	RATE = 5
X ← expr	x = expr
INPUT X	x = int(input("..."))
OUTPUT A, "is", B	print(A, "is", B) or f-string
IF c THEN ... ELSE ... ENDIF	if c: ... else: ...
nested IF inside ELSE	nested if inside else (or elif ladder)
CASE OF X ... ENDCASE	nested if statements (or elif ladder)
FOR I ← 1 TO N ... NEXT I	for i in range(1, N + 1):
FOR I ← A TO B STEP S	for i in range(A, B + 1, S):
loop over elements with index	for value in a: (for-each)
WHILE c DO ... ENDWHILE	while c:
REPEAT ... UNTIL c	while True: ... if c: break
Total ← Total + Value	total += value (also -=, *=, /=)
IF Ch = "A" OR Ch = "E" OR ...	if ch in "AEIOU":
manual total / max / min	sum(a) / max(a) / min(a)
OUTPUT "---" repeated	print("-" * 20)
OUTPUT "Total: ", T	print(f"Total: {T}")
reverse loop with STEP -1	s[::-1]
DIV(A, B)	A // B
MOD(A, B)	A % B
A ^ B	A ** B
X <> Y	X != Y
ARRAY[1:N] OF INTEGER	a = [0] * N (indexes 0 to N-1)
Grid[2, 3]	grid[1][2]
LENGTH(s)	len(s)
SUBSTRING(s, p, l)	s[p-1 : p-1+l]
UCASE(s) / LCASE(s)	s.upper() / s.lower()
ROUND(x, n) / RANDOM()	round(x, n) / random.random()
PROCEDURE P(x : INTEGER) ... CALL P(5)	def p(x): ... p(5)

Pseudocode	Python
FUNCTION F(x) RETURNS INTEGER ...	def f(x): return ...
// comment	# comment
OPENFILE "F" FOR READ / WRITE	open("F", "r") / open("F", "w")
READFILE "F", v	v = f.readline()
WRITEFILE "F", v	f.write(str(v))
CLOSEFILE "F"	f.close()

Appendix B — command words

Knowing what each command word demands is free marks:

Command word	What it means
Calculate	work out from given facts, figures or information
Compare	identify similarities and/or differences
Define	give a precise meaning
Demonstrate	show how, or give an example
Describe	state the points / give characteristics and main features
Evaluate	judge quality, importance, amount or value
Explain	give purposes or reasons; make relationships clear; say why/how
Give	produce an answer from a source or from memory
Identify	name, select or recognise
Outline	set out the main points
Show (that)	provide structured evidence leading to a given result
State	express in clear terms
Suggest	apply knowledge to a situation with a range of valid responses